

Plug & Pray

Documentación técnica del sistema — «donde el Scheduler propone y el Segfault dispone».

Recorrido archivo por archivo de la implementación en C, explicando **qué hace** cada pieza, **qué problema resuelve** dentro del sistema y **por qué está construida de esa manera**. Cierra con un resumen teórico que vincula el código con los conceptos de la materia.

Estado documentado: **Checkpoint 2 – Planificación**

Módulos: 6 + biblioteca compartida

Lenguaje: C (so-commons-library)

Arquitectura: distribuida por sockets TCP

CÓMO LEER ESTE DOCUMENTO

Cada módulo abre con su rol y su color. Dentro, cada archivo se presenta como una ficha con tres preguntas: **Qué hace** (responsabilidad), **Qué resuelve** (el problema del sistema que atiende) y **Por qué así** (la decisión de diseño detrás). Los archivos de *boilerplate* repetido — loggers y configs— se explican una vez como patrón y luego se listan sus diferencias por módulo, para no repetir lo mismo seis veces.

ÍNDICE

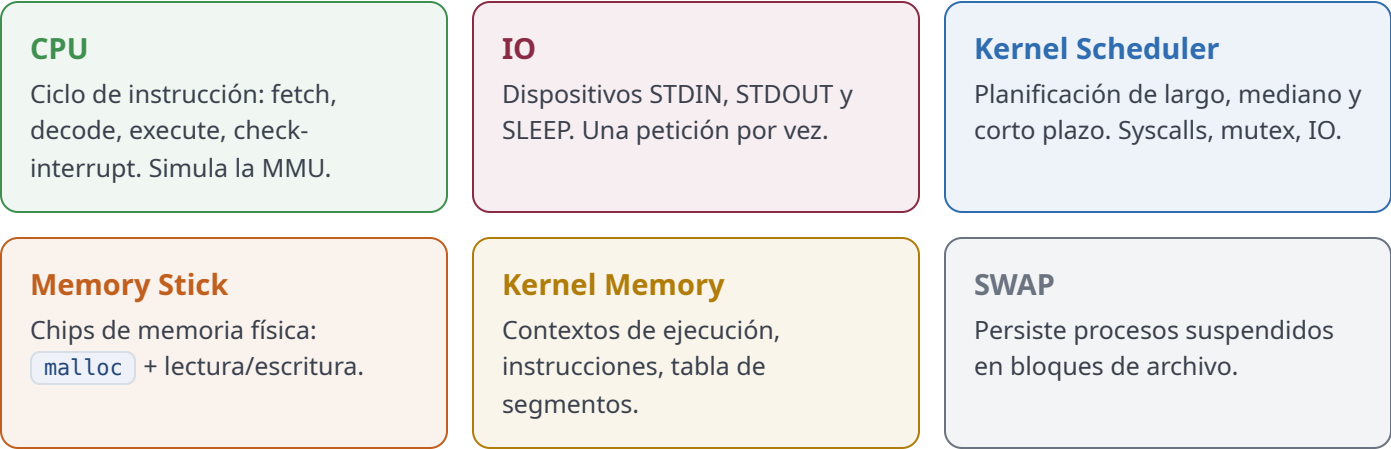
- | | |
|--|---------------------------------|
| 01 Panorama y arquitectura | 06 Módulo Kernel Memory |
| 02 Protocolo de comunicación | 07 Módulo CPU |
| 03 Patrones comunes (boot, config, logger) | 08 Módulo IO |
| 04 Biblioteca utils | 09 Módulo Memory Stick |
| 05 Módulo Kernel Scheduler | 10 Módulo SWAP |
| | 11 Resumen teórico y la materia |

01 · Panorama y arquitectura

Plug & Pray simula un sistema operativo distribuido: varios procesos reales de Linux —cada uno un módulo en C— que cooperan por sockets TCP para **planificar**

«Procesos» simulados, **ejecutar** su pseudocódigo y **administrar** su memoria bajo un esquema de segmentación.

El sistema modela la separación clásica de un SO real: quién *decide* qué corre (Kernel Scheduler), quién *ejecuta* instrucciones (CPU), quién *guarda* el estado y la memoria (Kernel Memory), quién *almacena físicamente* los datos (Memory Stick y SWAP) y quién *simula* la entrada/salida (IO). Al ser procesos independientes, el TP obliga a resolver de verdad los problemas centrales de la materia: comunicación, serialización, concurrencia y sincronización.



Las flechas de dependencia de la consigna se traducen en un patrón **cliente/servidor** muy claro dentro del código:

MÓDULO	ACTÚA COMO SERVIDOR DE...	ACTÚA COMO CLIENTE DE...
kernel_memory	CPU, Kernel Scheduler, Memory Stick, SWAP	—
kernel_scheduler	CPU, IO	Kernel Memory
memory_stick	CPU	Kernel Memory
cpu	—	Kernel Scheduler, Kernel Memory, Memory Stick
io	—	Kernel Scheduler
swap	—	Kernel Memory

¿Qué está implementado en este Checkpoint?

El repositorio está congelado en el **Checkpoint 2 (Planificación)**. Eso define qué código es «real» y qué es todavía un *mock* o un esqueleto, y es clave para entender por qué ciertos archivos están «a medio hacer»: no es una omisión, es exactamente el alcance que pedía la cátedra para esta fecha.

MÓDULO	ESTADO EN CP2	DETALLE
kernel_scheduler	funcional	Largo y corto plazo con FIFO/RR, estado BLOCK, colas de IO y mutex sin herencia de prioridades.
cpu	funcional	Ciclo de instrucción completo, decode/execute, MMU y syscalls. Interactúa con memoria vía mocks.
io	completo	STDIN, STDOUT y SLEEP operativos.
kernel_memory	mock	Devuelve instrucciones desde archivo, espacio libre fijo y responde OK a lecturas/escrituras sin memoria real.
memory_stick	esqueleto	Conexión y handshake; las operaciones de lectura/escritura quedan para CP3.
swap	esqueleto	Solo se conecta a Kernel Memory. Sin operaciones de bloque aún.

02 · Protocolo de comunicación

Antes de mirar módulo por módulo conviene entender el «idioma» que todos comparten, porque casi todos los archivos `conexiones.c` hablan este mismo protocolo.

Handshake numérico

Cuando un módulo se conecta a un servidor, lo primero que envía es un entero de identificación. El servidor lo lee y decide a qué rutina de atención derivar la conexión. Es el equivalente a «decir quién sos» al golpear la puerta:

CÓDIGO	QUIÉN SE PRESENTA
1	CPU
2	IO
3	Kernel Scheduler
4	Memory Stick
5	SWAP

Mensajes y serialización

Pasado el handshake, la conversación se estructura con un **código de mensaje** (el `enum t_codigo_mensaje` de `utils/protocolo.h`) seguido de los campos que ese mensaje necesita. La serialización es **directa por campos**: se envían enteros de tamaño fijo (`uint32_t`), structs completos (los registros de CPU, un segmento) y strings precedidos por su longitud. No hay un *buffer* genérico empaquetado; cada `send` tiene su `recv` espejo del otro lado.

POR QUÉ ESTE ESTILO DE SERIALIZACIÓN

Enviar campo por campo es más verboso que empaquetar todo en un `t_paquete`, pero para un protocolo con pocos mensajes es más transparente: leyendo el `send` y su `recv` se entiende el formato exacto del *wire*. El riesgo es la **desincronización**: si un lado manda un campo que el otro no lee (o en otro orden), el socket queda corrido para siempre. Por eso el código tiene comentarios insistiendo en mandar *primero el código y después el struct* (ver `enviar_contexto_a_ks` en CPU).

Los strings viajan con el patrón `[uint32_t longitud][bytes]`, y los datos binarios de tamaño fijo (como `t_registros_cpu`) se mandan con un único `send` del struct entero. Esto funciona porque todos los módulos compilan con la misma definición del struct desde `utils/registros.h` — un contrato compartido.

03 · Patrones comunes: boot, config y logger

Seis módulos comparten exactamente la misma estructura de arranque. Documentarla una vez evita repetir la explicación seis veces; después, en cada módulo, solo se señalan las diferencias reales (qué claves lee su config).

El trío `config` · `logger` · `main`

- `<modulo>_config.c/.h` — define un struct con los parámetros del módulo, verifica que el archivo `.config` tenga todas las claves obligatorias (`config_has_property`), copia los valores con `strdup` y ofrece `_create`, `_log` y `_destroy`. **Resuelve**: poder cambiar el comportamiento del módulo sin recompilar, requisito explícito de la consigna. **Por qué así**: valida al inicio y falla temprano si falta una clave, en vez de reventar más tarde con un `NULL`.
- `<modulo>_logger.c/.h` — crea dos loggers: uno de *boot* (nivel INFO fijo) para la fase de arranque, y el principal, cuyo nivel sale del config (`log_level_from_string`). **Por qué dos**: durante el arranque todavía no se leyó el `LOG_LEVEL` del config, así que el boot logger da un nivel conocido hasta que el principal esté listo.
- `main.c` — orquesta el arranque: boot logger → valida argumentos → carga config → crea logger principal → establece conexiones → entra al bucle de trabajo. Todos siguen la misma escalera de `if (x == NULL) { limpiar; return EXIT_FAILURE; }`, liberando lo ya creado en cada punto de falla.

DATO: HAY DOS MECANISMOS DE BOOT CONVIVIENDO

`utils` ofrece un `boot_loader` genérico (`system_boot`) pensado para centralizar este arranque, pero en la práctica cada módulo trae su propio par `config` / `logger` y **no** lo usa. Es una decisión que quedó a mitad de camino: la abstracción existe pero el equipo prefirió el control explícito por módulo. Vale tenerlo presente para el coloquio.

utils

El «contrato» que compilan todos los módulos: tipos comunes, códigos de protocolo y sockets. Se instala como biblioteca estática y se incluye con `<utils/...>`.

protocolo.h `utils/src/utils/`

QUÉ HACE

Define dos `enum` centrales: `t_codigo_mensaje` (HANDSHAKE, ENVIAR_PID, PEDIR_INSTRUCCION, SYSCALL, CONTEXTO, LEER/ESCRIBIR_MEMORIA, etc.) y `t_codigo_syscall` (SLEEP, STDIN, STDOUT, MUTEX_*, INIT_PROC, EXIT, MEM_*).

QUÉ RESUELVE

Que todos los módulos «hablen el mismo idioma»: un mismo entero significa lo mismo en los seis procesos.

POR QUÉ ASÍ

Al ser un `enum` compartido, agregar un mensaje nuevo es cambiar un solo archivo y recompilar todo. El orden de los valores es parte del contrato: no se deben reordenar sin recompilar todo.

protocolo.c `utils/src/utils/`

QUÉ HACE

Traduce cada código de mensaje/syscall a un string legible (`codigo_mensaje_to_string`, `codigo_syscall_to_string`).

QUÉ RESUELVE

Logs entendibles: en vez de «recibí un 4» el log dice «recibí FIN_IO».

POR QUÉ ASÍ

Es puro azúcar de logging, pero fundamental porque la consigna exige logs mínimos claros para que el TP sea evaluable.

sockets.c / sockets.h `utils/src/utils/sockets/`

QUÉ HACE

Dos funciones: `crear_servidor(puerto)` (socket + bind + listen) y `crear_conexion(ip, puerto)` (socket + connect), ambas resolviendo direcciones con `getaddrinfo`.

QUÉ RESUELVE

Encapsula el *boilerplate* de Berkeley sockets para que ningún módulo tenga que repetir la danza de `getaddrinfo` / `bind` / `listen`.

POR QUÉ ASÍ

Usa `SO_REUSEPORT` para poder reiniciar un servidor sin esperar el `TIME_WAIT` del puerto — comodidad enorme al debuggear. Libera siempre el `addrinfo` y cierra el fd ante error, evitando fugas de descriptores.

registros.h `utils/src/utils/`

QUÉ HACE

Define `t_registros_cpu`: PC (`uint32_t`), AX-DX (`uint8_t`), EAX-EDX/SI/DI (`uint32_t`), con los tamaños exactos que pide la consigna.

QUÉ RESUELVE

Es el **contexto de ejecución** compartido: la CPU lo usa, Kernel Memory lo guarda por proceso, y viaja por el socket como bloque binario.

POR QUÉ ASÍ

Que esté en `utils` permite mandar el struct completo con un solo `send`: como ambos lados lo compilan idéntico, el *layout* en memoria coincide y no hace falta serializar campo por campo.

segmento.h `utils/src/utils/`

QUÉ HACE

Define `t_segmento` con `id_segmento`, `base`, `limite` y `tamano`.

QUÉ RESUELVE

Representa una entrada de la tabla de segmentos, que Kernel Memory administra y la CPU necesita para que la MMU traduzca direcciones lógicas a físicas.

POR QUÉ ASÍ

Compartido para que la tabla de segmentos viaje entera de KM a CPU sin traducciones intermedias.

estados.h `utils/src/utils/`

QUÉ HACE

Enum `t_estado_proceso` con los 7 estados (NEW, READY, EXEC, BLOCK, SUSP_BLOCK, SUSP_READY, EXIT) y `estado_to_string` como `inline`.

QUÉ RESUELVE

Modela el diagrama de 7 estados que exige la consigna.

POR QUÉ ASÍ

El prefijo `ESTADO_` evita colisión con la macro estándar `EXIT` de `<stdlib.h>` — un detalle sutil pero real que rompería la compilación sin el prefijo.

instrucciones.c / instrucciones.h `utils/src/utils/`

QUÉ HACE

Define `t_codigo_instruccion` (el set de instrucciones), la estructura `t_instruccion` (código + hasta 3 parámetros de 64 chars), y helpers para volcarlas a string.

QUÉ RESUELVE

Es el resultado del *decode* de la CPU: una instrucción ya parseada y lista para ejecutar.

POR QUÉ ASÍ

Parámetros de tamaño fijo (`[3][64]`) en vez de punteros dinámicos: evita `malloc` / `free` por instrucción en el *hot path* del ciclo, a costa de un límite duro de 3 parámetros — suficiente para todo el set del TP.

boot_loader.c / boot_loader.h `utils/src/utils/boot_loader/`

QUÉ HACE

Ofrece `system_boot` y `validate_config_rules`: un arranque genérico que crea logger, carga config y valida claves obligatorias.

QUÉ RESUELVE

Pretende centralizar el patrón de arranque de *todos* los módulos en una sola función.

POR QUÉ ASÍ

Nota importante: hoy ningún módulo lo usa —cada uno trae su propio `config` / `logger`. Es infraestructura preparada pero no adoptada; una posible refactorización futura, no código muerto que moleste.

Kernel Scheduler

Decide qué Proceso corre y cuándo. Administra los 7 estados, atiende las syscalls que la CPU delega, gestiona los mutex y despacha la IO. Es el módulo con más concurrencia del TP.

```
→ ~ ./bin/kernel_scheduler [config] [path proceso inicial]
```

main.c kernel_scheduler/src/ **83 líneas**

QUÉ HACE

Arranca el módulo, se conecta a Kernel Memory (como cliente), crea el `t_planificador` central, lanza los **hilos de largo y corto plazo**, encola el **proceso inicial como PID 0** y finalmente levanta el servidor que escucha CPUs e IO.

QUÉ RESUELVE

El orden de arranque: primero conectar a memoria (dependencia dura de la arquitectura), después planificar, y recién ahí aceptar conexiones.

POR QUÉ ASÍ

El PID 0 se crea a mano con prioridad máxima porque es la semilla del sistema —de él nacen todos los demás vía `INIT_PROC`—. Los hilos de planificación se lanzan *antes* de encolar, así el proceso inicial se procesa apenas entra.

planificador.h .../planificador/ **137 líneas**

QUÉ HACE

Define el `t_planificador`: las 7 colas de estado, las listas de CPUs/IOs/mutex conectados, y toda la batería de sincronización — **un mutex por cada estructura compartida y semáforos** para procesos en NEW/READY, CPUs libres y cada tipo de IO libre.

QUÉ RESUELVE

Es la estructura de datos maestra del scheduler: todo el estado del sistema de planificación vive acá.

POR QUÉ ASÍ

Un mutex fino por estructura (no un candado global) permite que, por ejemplo, un hilo mueva algo de BLOCK a READY mientras otro toca la lista de CPUs, sin bloquearse mutuamente. Los semáforos convierten la espera activa en espera pasiva: el hilo de corto plazo *duerme* hasta que haya a la vez un proceso en READY y una CPU libre.

QUÉ HACE

Implementa toda la mecánica de planificación. Se puede leer en cinco bloques:

a) Planificador de largo plazo — hilo_planificador_largo_plazo

Espera en `sem_procesos_new`, saca un PCB de la cola NEW, le avisa a Kernel Memory que cree el proceso (le manda PID + path del pseudocódigo) y lo transiciona a READY. **Resuelve** la admisión de procesos al sistema. **Por qué así:** como los procesos «nacen sin memoria asignada», este pasaje no tiene límite y es casi directo; el único paso real es registrar el proceso en KM para que después pueda servir sus instrucciones.

b) Planificador de corto plazo — hilo_planificador_corto_plazo

El clásico READY → EXEC. Hace `sem_wait` sobre **dos** semáforos (hay proceso en READY y hay CPU libre), saca el PCB, busca una CPU libre, la marca ocupada, mueve el PCB a la lista de EXEC y le despacha el PID por socket. Si el algoritmo es **RR**, además lanza un hilo temporizador de quantum.

DETALLE DE DISEÑO FINO

El `mutex_cpus` se mantiene tomado *hasta después* de hacer el `send` del PID a la CPU. Esto es deliberado: evita que el hilo del timer de quantum o el de atención de syscalls escriban al mismo socket al mismo tiempo y corrompan el protocolo. Es el tipo de decisión que separa un TP que «anda a veces» de uno robusto.

c) Timer de quantum — hilo_timer_quantum

Para Round Robin: duerme el quantum configurado y, si el mismo PID sigue en la misma CPU, le manda `MENSAJE_INTERRUPCION`. **Por qué el chequeo:** si el proceso ya se bloqueó o terminó antes de que venza el quantum, no hay que interrumpir a quien sea que ocupe ahora esa CPU. Verifica `id_cpu` + `pid_ejecutando` antes de desalojar.

d) Transiciones de estado seguras

Un conjunto de funciones `planificador_transicionar_*` (NEW → READY, EXEC → EXIT, EXEC → READY, EXEC → BLOCK, BLOCK → READY) que **siempre** hacen lo mismo: tomar el mutex de las colas involucradas, mover el PCB, actualizar su estado y timestamp, loguear el cambio con el formato exacto de la consigna y postear el semáforo correspondiente. **Por qué centralizarlas:** cada transición es una sección crítica; tenerlas en una sola función por transición evita repetir el patrón de locking (y equivocarse) en cada *call site*.

El helper `remover_de_exec` y la búsqueda-y-reencolado en `transicionar_block_a_ready` resuelven un problema práctico de las `t_queue` de las commons: no tienen «remover por

criterio», así que se recorre la cola sacando y volviendo a meter todo salvo el que se busca.

e) IO asíncrona, contexto y mutex

`planificador_ejecutar_io_async` lanza un hilo por pedido de IO: toma un dispositivo libre del pool (bloqueándose en el semáforo del tipo), ejecuta la operación —SLEEP directo, o STDIN/STDOUT dialogando con Kernel Memory por el contenido— y al terminar transiciona el proceso de BLOCK a READY. **Por qué un hilo por pedido:** así el scheduler no se bloquea esperando que termine una IO lenta; puede seguir planificando otros procesos.

El bloque de **mutex** (`crear` , `lock` , `unlock`) implementa exclusión mutua sobre recursos nombrados, con cola de bloqueados FIFO. Al liberar, si hay alguien esperando, se le entrega el mutex y se lo despierta (BLOCK → READY) *en el orden en que lo pidieron* — tal como aclara la v1.1 de la consigna. Todavía **sin herencia de prioridades** (eso es CP3).

QUÉ HACE

El servidor multihilo del scheduler. `manejar_cliente` lee el handshake y deriva: si es CPU la registra y entra a `atender_peticiones_cpu`; si es IO la registra en el pool.

`atender_peticiones_cpu` es un bucle gigante que resuelve **cada syscall** que llega de una CPU.

QUÉ RESUELVE

Es el puente entre lo que la CPU «no puede hacer sola» (syscalls) y las acciones del kernel.

Traduce cada `SYSCALL_*` en transiciones de estado concretas.

POR QUÉ ASÍ

Un hilo por CPU permite atender varias CPUs en paralelo. El patrón repetido en cada syscall — recibir registros, actualizar contexto en KM, transicionar, liberar la CPU y postear

`sem_cpus_libres` — es intencional: **toda** syscall libera la CPU (como dice la consigna), la diferencia está solo en a qué estado va el proceso.

SYSCALL	QUÉ HACE EL SCHEDULER	DESTINO DEL PROCESO
<code>EXIT</code>	Finaliza el proceso, motivo SUCCESS	EXEC → EXIT
<code>SLEEP</code>	Lanza IO SLEEP asíncrona	EXEC → BLOCK → (READY)
<code>STDIN / STDOUT</code>	Lanza IO asíncrona con Kernel Memory de por medio	EXEC → BLOCK → (READY)
<code>MUTEX_CREATE</code>	Crea el mutex si no existía	EXEC → READY
<code>MUTEX_LOCK</code>	Toma el mutex, o encola al proceso si está tomado	READY (si libre) / BLOCK (si ocupado)
<code>MUTEX_UNLOCK</code>	Libera y despierta al siguiente en la cola	EXEC → READY
<code>INIT_PROC</code>	Genera un PID nuevo y encola el hijo en NEW	Padre → READY; hijo → NEW

También maneja `MENSAJE_INTERRUPCION` (fin de quantum): recibe los registros actualizados, los sincroniza con KM, loguea el desalojo y manda el proceso a READY.

pcb.c / pcb.h .../pcb/ 45 + 57 líneas

QUÉ HACE

Define el **Process Control Block**: PID, path, estado, prioridad (original y actual), CPU asignada, motivo/recurso de bloqueo, lista de mutex tomados, copia de registros y timestamps por transición.

QUÉ RESUELVE

Es la ficha que el kernel mantiene por cada Proceso; concentra todo lo que necesita para planificarlo.

POR QUÉ ASÍ

Guarda `prioridad_original` y `prioridad_actual` por separado ya pensando en la herencia de prioridades de CP3 (donde una sube temporalmente y hay que poder volver a la original). La `lista_mutex_tomados` permite saber qué recursos liberar si el proceso muere.

mutex_kernel.c / mutex_kernel.h .../mutex_kernel/

QUÉ HACE

Define `t_mutex_kernel`: nombre, PID dueño, flag `tomado` y una **cola de PIDs bloqueados**.

QUÉ RESUELVE

Modela un semáforo tipo mutex del «lenguaje de scripting» del TP: uno solo lo tiene, el resto espera en cola.

POR QUÉ ASÍ

La cola es una `t_queue` (FIFO), lo que garantiza el desbloqueo «en el orden en que solicitaron el mutex» exigido por la consigna.

io_scheduler.h · config.c/.h · logger.c/.h .../

QUÉ HACE

`io_scheduler.h` define `t_tipo_io` y `t_pedido_io`. El resto es el patrón config/logger ya descrito en §03.

CONFIG PROPIA

Lee `PLANIFICATION_ALGORITHM` (FIFO/RR/CMN), `QUEUES_ALGORITHMS` (lista, para CMN), `RR_QUANTUM`, `QUEUE_PREEMPTION`, `SUSPENSION_TIMEOUT`, más IP/puerto de escucha y de Kernel Memory.

Kernel Memory

Guarda el contexto de ejecución de cada Proceso (registros + tabla de segmentos), sirve las instrucciones del pseudocódigo y —a futuro— administra la memoria de usuario sobre Memory Sticks y SWAP. En CP2 la parte de memoria física está **mockeada**.

```
→ ~ ./bin/kernel_memory [config]
```

main.c kernel_memory/src/

QUÉ HACE

Carga config y logger, crea el **gestor de procesos** y levanta el servidor multihilo que atiende a todos (CPU, Scheduler, Memory Stick, SWAP).

POR QUÉ ASÍ

Kernel Memory es el **primer** módulo que debe estar arriba (todos dependen de él), por eso su **main** es puro «preparar estructuras y ponerse a escuchar».

conexiones.c ../conexiones/

QUÉ HACE

Servidor multihilo: por cada conexión, un hilo lee el handshake y deriva a **atender_cpu**, **atender_scheduler**, o (por ahora) loguea Memory Stick/SWAP como «pendiente».

QUÉ RESUELVE

Atención concurrente de peticiones, como pide la consigna: muchas CPUs pidiendo instrucciones a la vez.

POR QUÉ ASÍ

Guarda config, gestor y logger en punteros estáticos de archivo para que los hilos accedan sin pasarlos por cada llamada. Simple y suficiente para un único servidor.

kernel_memory_gestor_procesos.c/.h .../gestor_procesos/

QUÉ HACE

Mantiene la lista de procesos (`t_proceso_km` = contexto + path del archivo), protegida por un mutex. Ofrece agregar, buscar (con y sin lock) y `gestor_obtener_instruccion`, que abre el archivo de pseudocódigo y devuelve la línea número `pc`.

QUÉ RESUELVE

Es la «memoria de instrucciones» y el almacén de contextos que la consigna pide mantener por cada PID.

POR QUÉ ASÍ

Leer la instrucción abriendo el archivo y avanzando hasta la línea `pc` es un **mock consciente**: la consigna deja libre cómo obtener las instrucciones en CP2. Es O(n) por fetch, pero simple y correcto; optimizarlo (cachear en RAM) queda para después. Expone `_sin_lock` + `lock/unlock` para que `atender_cpu` pueda leer el contexto de forma atómica mientras arma la respuesta.

kernel_memory_atencion_cpu.c .../atencion_cpu/

QUÉ HACE

Apenas conecta una CPU le manda el `SEGMENT_MAX_SIZE` (que la CPU necesita para su MMU). Después atiende dos mensajes: `MENSAJE_CONTEXTO` (devuelve registros + tabla de segmentos del PID) y `MENSAJE_PEDIR_INSTRUCCION` (devuelve la línea del pseudocódigo tras esperar el `INSTRUCTION_DELAY`).

QUÉ RESUELVE

Alimenta el ciclo de instrucción de la CPU: sin este servicio la CPU no tendría ni contexto ni instrucciones que ejecutar.

POR QUÉ ASÍ

Mandar el `SEGMENT_MAX_SIZE` en el handshake resuelve elegantemente que la CPU conozca ese parámetro sin tener que leerlo de su propio config (donde no está). El `INSTRUCTION_DELAY` simula la latencia de acceso a memoria, requisito de la cátedra para ver la planificación «en cámara lenta».

kernel_memory_atencion_scheduler.c .../atencion_scheduler/

QUÉ HACE

Atiende al Scheduler: crear proceso (recibe PID + path, arma el path completo con `SCRIPTS_BASEPATH`), devolver espacio libre (valor fijo `FREE_SPACE MOCK`), leer/escribir memoria (responden OK sin memoria real) y actualizar contexto (guarda los registros del PCB desalojado).

QUÉ RESUELVE

Es el punto donde el Scheduler «materializa» sus decisiones en la memoria: dar de alta un proceso, mantener su contexto sincronizado, y —en el futuro— pedir datos de usuario.

POR QUÉ ASÍ

Lectura/escritura devuelven OK y un buffer en cero: es exactamente el `mock` que pide el Checkpoint 2 («contestar OK a lecturas y escrituras sin implementarlas»). `ACTUALIZAR_CONTEXTO` sí es real y crucial: sin él, al re-despachar un proceso la CPU pediría un contexto viejo y repetiría instrucciones.

contexto.h · memoria_instrucciones.h · config.c/.h · logger.c/.h

QUÉ HACE

`contexto.h` define `t_contexto_ejecucion` (PID + registros + tabla de segmentos). `memoria_instrucciones.h` declara la interfaz del mock de fetch. El resto es el patrón config/logger.

CONFIG PROPIA

`SEGMENT_MAX_SIZE`, `ALLOCATION_STRATEGY` (BEST/WORST), `INSTRUCTION_DELAY`, `COMPACTION_DELAY`, `SCRIPTS_BASEPATH` y `FREE_SPACE MOCK` (clave extra del equipo para el mock de espacio libre).

CPU

Simula una CPU real simplificada: pide instrucciones, las decodifica, las ejecuta y chequea interrupciones. Traduce direcciones lógicas a físicas con su MMU. Se pueden conectar y desconectar varias en caliente.

```
→ ~ ./bin/cpu [config] [identificador]
```

main.c `cpu/src/`

QUÉ HACE

Recibe su ID por argumento, se conecta a los tres módulos (Kernel Memory, Memory Stick, Kernel Scheduler), crea el `t_cpu` y entra a atender al Scheduler.

POR QUÉ ASÍ

El ID por parámetro permite distinguir logs y configs de cada CPU —requisito literal de la consigna—. Recibe el `segment_max_size` durante el handshake con KM y se lo guarda al `t_cpu` para la MMU.

cpu.c / cpu.h `.../cpu/` 127 líneas

QUÉ HACE

Define el estado `t_cpu` (registros, sockets, PID actual, tabla de segmentos) y `atender_peticiones_kernel_scheduler`: cuando el Scheduler manda un PID, la CPU pide su contexto y entra al **bucle del ciclo de instrucción** hasta que el proceso se libera.

QUÉ RESUELVE

Orquesta el ciclo Fetch → Decode → Execute → Check-Interrupt que es el núcleo de todo el módulo.

POR QUÉ ASÍ

El manejo del **Program Counter** es el punto más delicado: se incrementa *después* de ejecutar (salvo que la instrucción fuera un `JNZ` que ya lo modificó). El código lo comenta explícitamente: si no se incrementa antes de mandar el contexto en una interrupción, al reanudar el proceso re-ejecutaría la misma instrucción en un **loop infinito**. Ese bug sutil está prevenido a propósito.

instrucciones.c .../instrucciones/ 302 líneas

QUÉ HACE

Implementa las cuatro etapas del ciclo más la MMU:

- **MMU** (`mmu_traducir`): dado `dir_logica`, calcula `num_segmento = dir/segment_max_size` y `desplazamiento = dir % segment_max_size`, busca el segmento en la tabla y devuelve `base + desplazamiento`. Si el segmento no existe o el acceso se pasa del límite, retorna `UINT32_MAX` como señal de **SEG_FAULT**.
- **Fetch**: le pide a Kernel Memory la instrucción del `PC` actual.
- **Decode**: parte el string por espacios (`string_split`), mapea el mnemónico a su código y copia los parámetros. Un comentario explica por qué corta apenas encuentra el `NULL` del arreglo — leer de más pasaría el final del array que devuelve `string_split`.
- **Execute**: un `switch` por instrucción. Las aritméticas/registros (NOOP, SET, SUM, SUB, JNZ) se resuelven en la CPU. MOV_IN/MOV_OUT/COPY_MEM usan la MMU y hablan con el Memory Stick. Las *syscalls* (SLEEP, IO, MUTEX_*, MEM_*, INIT_PROC, EXIT) delegan al Scheduler y devuelven `1` para cortar el ciclo (liberan la CPU).
- **Check-Interrupt**: con `MSG_DONTWAIT` mira si el Scheduler mandó una interrupción sin bloquearse. Si la hay, manda el contexto de vuelta y corta.

POR QUÉ ASÍ

Ante SEG_FAULT, cualquier instrucción de memoria dispara `enviar_syscall_exit` y termina el proceso — modelando que un acceso inválido mata al proceso, como en un SO real. El `MSG_DONTWAIT` del check-interrupt es la clave para que el chequeo no frene el ciclo cuando no hay interrupción pendiente.

registros_cpu.c / registros_cpu.h .../registros_cpu/

QUÉ HACE

Get/set de registros por nombre, en dos anchos: `get/set_registro` (8 bits, AX-DX) y `get/set_registro_32` (32 bits, EAX-EDX/SI/DI/PC).

QUÉ RESUELVE

Que el *execute* pueda operar registros a partir del string del parámetro (`"AX"`, `"EAX"` ...) sin un `if` gigante en cada instrucción.

POR QUÉ ASÍ

La versión de 32 bits también acepta los de 8 bits (los promueve a `uint32_t`): útil para instrucciones como `JNZ` que pueden mirar cualquier registro sin importar su ancho.

`conexiones.c` / `conexiones.h` .../conexiones/ 206 líneas

QUÉ HACE

Establece las tres conexiones y contiene **todos los emisores de syscalls** (`enviar_syscall_*`), el envío de contexto al Scheduler, el pedido de contexto a KM y la lectura/escritura al Memory Stick.

QUÉ RESUELVE

Concentra en un solo archivo cómo la CPU habla con el resto del sistema — el *execute* solo llama funciones con nombres claros.

POR QUÉ ASÍ

El helper interno `enviar_syscall_a_ks` hace lo común a toda syscall: **incrementa el PC en 1** y manda el código. Ese incremento centralizado garantiza que, al volver de la syscall, el proceso continúe en la instrucción siguiente y no repita la llamada. Un comentario en `enviar_contexto_a_ks` advierte que hay que mandar primero el código de mensaje y después el struct, o el socket se desincroniza para siempre.

IO

Simula un dispositivo de E/S. El tipo (STDIN, STDOUT o SLEEP) se elige por argumento al arrancar. Un solo hilo, una petición por vez — igual que un dispositivo real que atiende en serie.

```
→ ~ ./bin/io [config] [STDIN|STDOUT|SLEEP]
```

main.c io/src/ 192 líneas

QUÉ HACE

Se conecta al Scheduler (mandando su nombre de interfaz en el handshake) y entra a un `switch` por tipo con el bucle de atención:

- **STDIN:** recibe PID y tamaño, le pide al usuario que ingrese caracteres por teclado, y devuelve exactamente `tamaño` bytes — recortando si sobra o rellenando con `'\0'` si falta.
- **STDOUT:** recibe PID, tamaño y el contenido; lo imprime por pantalla y en el log; devuelve OK.
- **SLEEP:** recibe PID y milisegundos, hace `usleep(ms*1000)` y devuelve OK.

QUÉ RESUELVE

Modela las tres formas de E/S del sistema y, sobre todo, el hecho de que la IO **bloquea** al proceso (pasa a BLOCK) mientras dura.

POR QUÉ ASÍ

El relleno/recorte a tamaño fijo en STDIN respeta el contrato con Kernel Memory: siempre devuelve el bloque del tamaño pedido. El `usleep` recibe microsegundos, de ahí el `×1000` — detalle que la propia consigna marca al pie. Un solo hilo por diseño: la consigna aclara que los módulos IO tienen un único hilo de ejecución.

io_tools.h · conexiones.c/.h · config.c/.h · logger.c/.h

QUÉ HACE

`io_tools.h` define `t_tipo_dispositivo_io` y el estado `t_io`. `conexiones.c` hace el handshake enviando además el nombre de la interfaz (para que el Scheduler sepa qué tipo de IO es). El resto, patrón config/logger.

CONFIG PROPIA

Solo `LOG_LEVEL` e IP/puerto del Kernel Scheduler — el módulo más liviano del TP.

Memory Stick esqueleto en CP2

Representa un chip de memoria: reserva un bloque con `malloc` y atiende lecturas/escrituras de CPU y Kernel Memory sobre él. En CP2 solo está el andamiaje de conexión.

```
→ ~ ./bin/memory_stick [config] // [tamaño] llega por config en esta versión
```

main.c `memory_stick/src/`

QUÉ HACE

Se conecta a Kernel Memory (como cliente) y levanta su propio servidor para recibir CPUs.

QUÉ RESUELVE

Prepara la topología: el Stick es a la vez cliente de KM (para informar su tamaño) y servidor de las CPUs (que le piden datos).

POR QUÉ ASÍ

El doble rol (cliente + servidor) es exactamente el que pide la arquitectura. La lógica de lectura/escritura sobre el bloque reservado queda explícitamente para el Checkpoint 3.

conexiones.c `.../conexiones/`

QUÉ HACE

Servidor multihilo que acepta CPUs (handshake), y cliente que se presenta ante KM. Hoy, al recibir una CPU, la registra y cierra —con un `TODO` marcando dónde irá la atención real.

POR QUÉ ASÍ

Dejar el `TODO` visible documenta el límite del checkpoint: la infraestructura de red está probada, falta la lógica de datos.

config.c/.h · **logger.c/.h**

CONFIG PROPIA

`LOG_LEVEL`, `MEMORY_DELAY` (latencia simulada), IP/puerto de escucha e IP/puerto de Kernel Memory.

SWAP

esqueleto en CP2

Persistirá en un archivo la memoria de los procesos suspendidos, dividida en bloques. En CP2 solo se conecta a Kernel Memory.

```
→ ~ ./bin/swap [config]
```

main.c · conexiones.c swap/src/

QUÉ HACE

Arranca, carga config, se conecta a Kernel Memory con su handshake (código 5) y termina. Aún no atiende operaciones de bloque.

QUÉ RESUELVE

Valida que el módulo compila, arranca y se integra a la red del sistema — el objetivo de conectividad del Checkpoint 1.

POR QUÉ ASÍ

La consigna aclara que SWAP «no debe tener estructura administrativa más allá de leer/escribir el archivo»: la inteligencia (qué bloque es de qué segmento) vive en Kernel Memory. Por eso este módulo será deliberadamente «tonto» incluso cuando esté completo.

config.c/.h · logger.c/.h

CONFIG PROPIA

LOG_LEVEL, SWAP_FILE_PATH, SWAP_FILE_SIZE, BLOCK_SIZE e IP/puerto de Kernel Memory.

Resumen teórico

Cada mecanismo del TP es la implementación concreta de un concepto de la teoría. Esta sección los enlaza, con referencia a los capítulos del apunte «*Sistemas Operativos for Dummies*» (páginas entre paréntesis) para poder ir del código al libro y del libro al código.

La idea rectora del TP es la que abre el apunte: un SO es un **administrador de recursos** que reparte CPU, memoria y E/S entre procesos que compiten. Plug & Pray no explica esa idea: la *construye*. Cada módulo es uno de esos administradores, y hacerlos funcionar juntos obliga a atravesar de punta a punta el programa de la materia.

Repaso de arquitectura → el módulo CPU (pág. 3–8)

El apunte arranca con los **registros** (PC, IR, MAR, de propósito general) y el **ciclo de instrucción** Fetch → Decode → Execute con su `PC = PC + 1`. El módulo CPU es ese diagrama hecho código:

`t_registros_cpu` tiene el mismo PC y registros de propósito general que describe la teoría, y `cpu.c` ejecuta literalmente las etapas del ciclo. Cuando el apunte habla del **ciclo con interrupciones** y la etapa «Check Interrupt» donde se pregunta si llegó una interrupción *después* de ejecutar la instrucción, el TP lo replica en `cpu_check_interrupt`: el fin de quantum es exactamente una **interrupción de clock** (pág. 5) que fuerza el desalojo para permitir multiprogramación.

CONCEPTO CLAVE QUE EL TP HACE TANGIBLE

El **contexto de ejecución** (guardar todos los registros al interrumpir un proceso, pág. 6) deja de ser abstracto: es el struct `t_registros_cpu` que viaja por el socket de la CPU al Scheduler y de ahí a Kernel Memory, y que se restaura al re-despachar. El «cambio a modo kernel» del apunte es, en el TP, **la syscall que la CPU delega al Scheduler**.

Procesos → el PCB y los 7 estados (pág. 14–23)

La teoría define el **Process Control Block** como la estructura donde el SO guarda todo lo de un proceso, y el modelo de estados con sus transiciones. El TP lo materializa en `t_pcb` y en `t_estado_proceso` (los 7 estados: NEW, READY, EXEC, BLOCK, SUSP_BLOCK, SUSP_READY, EXIT). Las funciones

`planificador_transicionar_*` son las flechas del diagrama de estados, una por una. La distinción teórica entre **suspendido** y **bloqueado** (un proceso bloqueado que además fue sacado de memoria) aparece como los estados SUSP, que entrarán en juego con el planificador de mediano plazo en el próximo checkpoint.

Planificación → largo, mediano y corto plazo (pág. 24–37)

Es el capítulo central de este checkpoint. La teoría distingue tres planificadores según su horizonte, y el TP los separa en hilos distintos:

EN LA TEORÍA	EN EL TP
Largo plazo (admisión, NEW → READY)	<code>hilo_planificador_largo_plazo</code>
Mediano plazo (suspensión, BLOCK ↔ SUSP)	diseñado; se completa en CP3
Corto plazo (dispatcher, READY → EXEC)	<code>hilo_planificador_corto_plazo</code>

Los **algoritmos** del apunte —FIFO y Round Robin— están implementados y son elegibles por config. El **quantum** de RR (el «tajo de tiempo» que la teoría asocia a la interrupción de clock) es `hilo_timer_quantum`. Las **Colas Multinivel** con desalojo entre prioridades quedan planteadas (`QUEUES_ALGORITHMS` , `QUEUE_PREEMPTION`) para CP3. Conceptos como *starvation* y *preemption* dejan de ser palabras: se ven en los logs cuando un proceso es «Desalojado por fin de quantum».

Arquitectura de Kernel → syscalls y modo dual (pág. 44–49)

El apunte explica que ciertas operaciones son **privilegiadas** y solo las puede hacer el SO: para eso existe la **llamada al sistema**. El TP lo modela con fidelidad sorprendente: la CPU *no puede* dormir un proceso, tomar un mutex o crear otro proceso por su cuenta; emite un `MENSAJE_SYSCALL` y **libera la CPU** hasta que el kernel resuelva. Esa frontera entre «lo que hace la CPU sola» (aritmética, registros) y «lo que requiere al kernel» (todo lo demás) es la separación usuario/kernel del apunte, llevada a la topología de red del sistema.

Concurrencia y sincronización → mutex y semáforos (pág. 50–64)

Este capítulo es el que más se «siente» al programar el TP. El apunte presenta la **sección crítica**, el **mutex** y el **semáforo**; el planificador los usa a dos niveles:

- **Sincronización interna:** los `pthread_mutex_t` (uno por cola) protegen las estructuras compartidas entre los muchos hilos del Scheduler, y los `sem_t` coordinan «hay proceso listo» + «hay CPU libre». Es teoría de concurrencia aplicada al propio SO simulado.
- **Los mutex del «lenguaje de scripting»** (`t_mutex_kernel`): el SO ofrece mutex a los Procesos simulados, con cola de espera FIFO. Es el mismo concepto, un nivel más arriba.

La **inversión de prioridades** y su solución, la **herencia de prioridades** (un concepto que la teoría trata junto a la sincronización), están declaradas en el PCB (`prioridad_original` vs `prioridad_actual`) pero todavía no implementadas — es el gran pendiente de CP3.

Deadlock → el riesgo latente (pág. 65–71)

Aunque el TP no pide detección de *deadlock*, el peligro es real y el diseño lo tiene presente: con varios mutex tomados por varios procesos podría formarse un abrazo mortal. La `lista_mutex_tomados` del PCB es la base para razonar sobre qué recursos retiene cada proceso. Además, en la concurrencia *interna*, el orden cuidadoso en que se toman los `pthread_mutex` (y los comentarios que lo justifican) es precisamente para evitar un deadlock entre los hilos del propio Scheduler.

Memoria Real → segmentación y asignación (pág. 72–85)

El esquema de memoria del TP es **segmentación pura**, uno de los modelos del apunte. La **traducción de dirección lógica a física** —que la teoría muestra en binario con número de segmento + desplazamiento— es `mmu_traducir`, hecha en decimal por comodidad pero con la misma fórmula. De aquí salen también:

- El **Segmentation Fault**: acceder fuera del límite del segmento (concepto de *protección de memoria*, pág. 72+) mata el proceso — implementado como el retorno `UINT32_MAX` de la MMU.
- La **fragmentación externa** y su remedio, la **compactación**, junto a los algoritmos de asignación **Best Fit / Worst Fit** (`ALLOCATION_STRATEGY`): están en la config y la consigna, y se implementan de lleno en CP3, cuando la memoria deje de estar mockeada.

Entrada/Salida → el módulo IO y el bloqueo (pág. 122+)

La teoría explica que la E/S es órdenes de magnitud más lenta que la CPU, y que por eso un proceso que pide E/S se **bloquea** y cede el procesador. Ese es exactamente el ciclo del TP: una syscall de IO manda el proceso a BLOCK, se ejecuta la E/S en un hilo aparte, y al terminar el proceso vuelve a READY. El `MEMORY_DELAY`, el `INSTRUCTION_DELAY` y el `SLEEP` son la forma del TP de hacer visible esa diferencia de velocidades que en hardware real es invisible de tan rápida.

LA BISAGRA TEORÍA-PRÁCTICA PARA EL COLOQUIO

La consigna es explícita: en el coloquio se evalúa «la relación de lo visto en la teoría con lo implementado», y una implementación que contradiga la teoría es motivo de desaprobación. Por eso este documento no describe solo *qué* hace cada archivo, sino *por qué* está hecho así — porque ese «por qué» casi siempre es un concepto de la materia tomando forma en C.

Fuera del alcance del TP (pero en el apunte)

Los capítulos de **Memoria virtual y paginación** (pág. 86–98) y **File System / FAT / EXT2** (pág. 99–121) no se implementan en Plug & Pray: el TP usa segmentación pura y no simula un sistema de archivos. Se mencionan para cerrar el mapa de la materia: son el contexto teórico que explica por qué el TP eligió segmentación (más simple de ver «a ojo» en decimal) y por qué la memoria secundaria del TP es un SWAP por bloques y no un file system completo.

Plug & Pray · Documentación técnica generada sobre el estado **Checkpoint 2** del repositorio.

Referencias: consigna «Plug & Pray» v1.1 (1C 2026) · apunte «Sistemas Operativos for Dummies» · docs.utnso.com.ar · [so-commons-library](#).